Here is a refined, highly professional version. It removes the conversational tone and metaphors, focusing strictly on system capabilities, architecture, and engineering execution.

````
# Scalable Distributed Architecture for Real-Time Clinical Genomics AI

This repository contains the implementation of a distributed big data ecosystem
engineered for the real-time processing and analysis of clinical genomic data.
Designed to overcome the computational bottlenecks of traditional
bioinformatics pipelines, the system leverages a microservices architecture to
ingest continuous patient data, run parallelized machine learning models, and
compute sequence divergence at scale.

**Author:** Mohamed Abadine
**Domain:** Distributed Systems, Data Engineering, Bioinformatics

---

## 🛠️ System Architecture & Infrastructure

The pipeline is containerized via Docker and designed for high throughput and
fault tolerance.

```text
[ Genomic Sequencer (Kafka Producer) ] ➔ [ Apache Spark Cluster (8 Cores) ] ➔
[ MongoDB ] ➔ [ Streamlit UI ]
                                                    ⬆
                                        [ Hadoop HDFS (Data Lake) ]
````

- **Ingestion Layer:** Apache Kafka & ZooKeeper orchestrate real-time telemetry from sequencing hardware.
- **Compute Layer:** Apache Spark (Standalone) handles micro-batch stream processing and distributed matrix factorization.
- **Storage Layer:** Hadoop Distributed File System (HDFS) stores historical `.parquet` datasets and serialized ML models.
- **Persistence Layer:** MongoDB provides high-velocity NoSQL document appends for processed clinical records.
- **Application Layer:** Streamlit serves as the reactive clinical diagnostic dashboard.

🔬 Core Analytical Engines

The project features two parallel processing engines for comprehensive genetic analysis:

1. Collaborative Filtering for Gene-Disease Association (ALS)

Designed to predict specific pathologies triggered by mutated genes based on historical data.
````

- **Dataset:** The Open Targets Platform (a highly sparse bipartite graph of established Gene-Disease associations).
- **Algorithm:** PySpark's **Alternating Least Squares (ALS)** natively distributes the matrix factorization across the Spark cluster, transforming the sparse interaction matrix into dense latent embeddings.
- **Performance:** Evaluated via an 80/20 train-test split on 1,586,005 valid rows, achieving a **Root Mean Square Error (RMSE) of 0.0638**. The model is serialized to HDFS for low-latency production inference.

2. Information-Theoretic Sequence Divergence (NCD)

An alignment-free algorithmic approach to measuring structural DNA mutations against a healthy reference sequence (BRCA1).

- **Algorithm:** Applies the **Burrows-Wheeler Transform (BWT)** via memory-efficient Suffix Arrays to achieve $\mathcal{O}(N)$ spatial complexity, followed by dictionary compression (`zlib`).
- **Clinical Logic:** Computes the **Normalized Compression Distance (NCD)** between the reference and patient sequences. The pipeline enforces diagnostic thresholds (Score < 0.1 for benign variation; Score > 0.4 for severe divergence).

🚀 Deployment & Execution

The architecture is deployed via `docker-compose`. Once the cluster is initialized, distributed jobs can be submitted directly to the Spark master.

Submit Batch Training (ALS Model)

Executes the data cleaning, model training, and HDFS serialization pipeline.

```
docker exec -it spark-master /spark/bin/spark-submit \
  --packages org.apache.spark:spark-sql-kafka-0-
10_2.12:3.3.0,org.mongodb.spark:mongo-spark-connector_2.12:3.0.1 \
  /spark_jobs/batch_train_model.py
```

Execute Compression Engine (Stress Testing)

Runs the BWT/NCD algorithms. Accepts a dynamic data fraction argument (e.g., `0.5` for 50%) to scale the payload and test cluster memory efficiency.

```
docker exec -it spark-master /spark/bin/spark-submit \
  /spark_jobs/compression_distance.py 0.5
```

Launch the Diagnostic Dashboard

Initializes the Streamlit UI for real-time patient monitoring.

```
streamlit run app.py
```

```

```